KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade
Anand Nagar, Krishnankoil, Srivilliputtur, Virudhunagar (Dt.) - 626126 | OMR, Chennai, Tamil Nadu
info@kalasalingam.ac.in | www.kalasalingam.ac.in

School of Computing

Department of Computer Science & Engineering

CODING BATTLE PLATFORM

Interactive Competitive Java Programming, Timed Battles & Faculty Command System

Mini Project Report

Submitted by

9925051090 – V. Uday Kumar Reddy (Lead)

9925051001 – K. Rahul Sharma

9925051017 – M. Ananya Verma

9925051033 – S. Kiran Patel

In partial fulfilment for the award of the degree of

Bachelor Of Technology

in

Computer Science and Engineering

SCHOOL OF COMPUTING COMPUTER SCIENCE AND ENGINEERING
KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION

KRISHNANKOIL 626 126

November 2026

KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade
Anand Nagar, Krishnankoil, Srivilliputtur, Virudhunagar (Dt.) - 626126 | OMR, Chennai, Tamil Nadu
info@kalasalingam.ac.in | www.kalasalingam.ac.in

DECLARATION

We hereby declare that this mini project **"CODING BATTLE PLATFORM"** is our genuine work carried out by us under the guidance of **Mrs. S. Reshni, ME.**, Assistant Professor, Department of Computer Science and Engineering, Kalasalingam Academy of Research and Education.

We declare that this project has been developed as part of the academic requirements for the **Object Oriented Programming using Java (CSE203)** course. The work presented in this report is our own, and all sources of information, references, and resources used in the development of the project have been duly acknowledged. We further declare that we have contributed to the design, development, implementation, testing, and documentation of the project according to our assigned roles. We take responsibility for the authenticity of the work submitted and confirm that this project has not been submitted previously, either in whole or in part, for any other academic assessment.

REGISTER NUMBER	STUDENT NAME	STUDENT SIGNATURE
9925051090	V. UDAY KUMAR REDDY	
9925051001	K. RAHUL SHARMA	
9925051017	M. ANANYA VERMA	
9925051033	S. KIRAN PATEL	

Date of End Semester Project Review: _____

KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade
Anand Nagar, Krishnankoil, Srivilliputtur, Virudhunagar (Dt.) - 626126 | OMR, Chennai, Tamil Nadu
info@kalasalingam.ac.in | www.kalasalingam.ac.in

BONAFIDE CERTIFICATE

Certified that this report titled **"Coding Battle Platform"**, for the Project Based Learning, is a Bonafide work of **V. UDAY KUMAR REDDY (9925051090)**, **K. RAHUL SHARMA (9925051001)**, **M. ANANYA VERMA (9925051017)**, and **S. KIRAN PATEL (9925051033)** who carried out the work together as a team for the fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**, during the Academic year Odd Semester (2026-2027).

Place: Krishnankoil

Date: _____

Mrs. S. Reshni, ME.,
Project Guide & Assistant Professor,
Department of Computer Science & Engineering,
Kalasalingam Academy of Research & Education,
Krishnankoil - 626126.

Dr. R. Raja Subramanian, M.E., Ph.D.,
Professor and Head of Department,
Department of Computer Science & Engineering,
Kalasalingam Academy of Research & Education,
Krishnankoil - 626126.

ACKNOWLEDGEMENT

We express our sincere gratitude to our institution and the **Department of Computer Science and Engineering** for providing us with the opportunity, advanced laboratories, and resources to undertake this mini project as part of the **Object Oriented Programming using Java (CSE203)** course.

We extend our heartfelt thanks to our Head of the Department, **Dr. R. Raja Subramanian, M.E., Ph.D.**, for the constant encouragement, academic facilities, and continuous support provided throughout the project lifecycle.

We are profoundly grateful to our course faculty and project guide **Mrs. S. Reshni, ME., Assistant Professor**, for her valuable guidance, technical suggestions, constructive reviews, and continuous mentorship during the design, development, sandboxed engine implementation, and testing of this project.

We also thank our classmates, peer reviewers, and faculty evaluation panel for their insightful feedback, cooperation, and teamwork which significantly enriched the platform's robustness.

Finally, we express our gratitude to our parents, family members, and friends who directly or indirectly encouraged and supported us in completing this mini project successfully.

TABLE OF CONTENTS

CHAPTER NO	CONTENTS	PAGE NO
1.	**Introduction**	**7**
	1.1 Problem Statement	7
	1.2 Objectives of the Project	7
	1.3 Scope of the Project	8
	1.4 Overview of the Coding Battle Platform	8
2.	**Existing & Proposed System**	**9**
	2.1 Existing System	9
	2.2 Limitations of the Existing System	9
	2.3 Proposed System	9
	2.4 Advantages of the Proposed System	10
	2.5 Comparison of Existing and Proposed System	10
3.	**Requirements**	**11**
	3.1 Hardware Requirements	11
	3.2 Software Requirements	11
	3.3 Functional Requirements	11
	3.4 Non-Functional Requirements	12
	3.5 Technologies Used	12
4.	**System Architecture & Design**	**13**
	4.1 System Architecture	13
	4.2 System Modules	13
	4.3 Application Workflow	14
	4.4 System Flowchart	15
	4.5 Database Design Overview (ERD)	16
5.	**OOP Concepts Used**	**17**
	5.1 Classes and Objects	17
	5.2 Encapsulation	17
	5.3 Abstraction	17
	5.4 Inheritance	17
	5.5 Polymorphism	17
	5.6 Application of OOP Concepts in the Project	18
6.	**Exception Handling & User-Defined Exceptions**	**19**
	6.1 Exception Handling in Java	19
	6.2 Exception Handling in the Project	19
	6.3 Built-in Exceptions	19
	6.4 User-Defined Exceptions	19
	6.5 Error Handling and Input Validation	20
7.	**Collections & Data Structures / AI**	**21**
	7.1 Java Collections Framework	21
	7.2 Custom Data Structures Used in the Project	21
	7.3 Data Management Using Java Objects	22
	7.4 AI & Machine Learning Applicability	22

TABLE OF CONTENTS (CONTD.)

CHAPTER NO	CONTENTS	PAGE NO
8.	**JDBC & Database**	**23**
	8.1 Introduction to JDBC	23
	8.2 Database Connectivity	23
	8.3 Database Schema and Tables	23
	8.4 Database Operations (CRUD)	24
	8.5 Prepared Statements	24
	8.6 Database Initialization and Failover Management	24
9.	**Graphical User Interface**	**25**
	9.1 User Interface Design	25
	9.2 Authentication Gate & Home Page	25
	9.3 Problem Solving Arena & Code Editor	25
	9.4 Timed Battle Arena & Countdown Clocks	25
	9.5 Global Leaderboard & Badges	26
	9.6 Faculty Command Suite & Assignment Broadcast	26
10.	**GitHub Details & Implementation**	**27**
	10.1 GitHub Repository Details	27
	10.2 Project Structure and Package Organization	27
	10.3 Important Classes and Their Responsibilities	27
	10.4 Implementation of Major Modules	28
	10.5 Database Configuration and Setup	28
	10.6 Steps to Run the Application	29
11.	**Testing**	**30**
	11.1 Testing Methodology	30
	11.2 Test Cases	30
	11.3 Functional Testing	30
	11.4 Database & Sandbox Testing	31
	11.5 Integration Testing	31
	11.6 Test Results	31
12.	**Results & Screenshots**	**32**
	12.1 Project Results & Platform Demonstration	32
	12.2 High-Definition Application Screenshots	33
13.	**Conclusion**	**41**
	13.1 Project Summary	41
	13.2 Learning Outcomes	41
14.	**Future Enhancement**	**42**
	14.1 AI-Based Automated Code Review	42
	14.2 Multi-Language Sandboxed Execution	42
	14.3 Distributed Proctoring & Plagiarism Detection	42

1. INTRODUCTION

In undergraduate computer science education, practical coding proficiency is paramount. Traditional laboratory curricula in Core Java often rely on offline coding, manual faculty inspections, paper records, and generic competitive programming websites. These traditional approaches fail to offer continuous automated feedback, lack integration with institutional syllabi, and provide no real-time synchronization during live classroom coding battles.

The **Coding Battle Platform** is an institutional competitive programming and academic laboratory evaluation system developed in accordance with the **Object Oriented Programming using Java (CSE203)** curriculum. It provides a full-stack Java architecture integrating **JavaFX 21** as the interactive frontend, **Core Java 17+** as the application layer incorporating custom Data Structures & Algorithms (DSA), and **JDBC** for persistent storage in a **MySQL** database (with resilient embedded H2 auto-failover).

1.1 Problem Statement

Computer science laboratories currently face three major bottlenecks: delayed manual evaluation of student source code, lack of process-isolated sandboxes leading to system freezes during infinite loops, and the absence of a unified faculty command portal to dispatch curriculum-aligned coding exams and monitor student cohorts in real-time. A unified, secure, and resilient platform that automatically compiles, sandboxes, evaluates, and ranks student Java programs is needed.

1.2 Objectives of the Project

1. To engineer an interactive Java competitive programming system utilizing JavaFX for desktop clients and modern web companion interfaces.
2. To design an isolated execution sandbox utilizing dynamic compilation (`javac`) and process watchdogs enforcing hard 3000ms execution timeouts.
3. To implement custom fundamental Data Structures and Algorithms (Max-Heap, QuickSort, Dual-Stack LIFO buffer, FIFO submission queue) without relying purely on standard collection black-boxes.
4. To build a dedicated Faculty Command Suite allowing instructors to schedule assignments, lab practicals, and exams, automatically broadcasting notifications to all students.
5. To maintain persistent storage through JDBC with parameterized SQL, ensuring zero viva downtime via auto-failover between MySQL and embedded H2.

1.3 Scope of the Project

The Coding Battle Platform encompasses the following functional domains:

- **Student Practice Arena:** Chronologically ordered Core Java problems covering Class/Object encapsulation, Inheritance, Polymorphism, Recursion, Collections, and DSA.
- **Timed Battle Arena:** Live synchronized countdown clock (MM:SS) with problem ribbons, instant solve indicators (✓), and dynamic scoreboard updates.
- **Process-Isolated Sandbox:** Security token AST scanning filtering illegal calls (`Runtime.getRuntime()`, `ProcessBuilder`, `System.exit()`), followed by isolated subprocess evaluation.
- **Faculty / Admin Command Suite:** Unified administrative portal for problem authoring, test case management, assignment dispatch, real-time broadcasting, and grade overrides.
- **Gamification & Ranking:** Algorithmic Top-K leaderboard sorting, category mastery progress bars, coding streaks, and milestone badges.

Out of Scope: External payment gateways, commercial SMS gateways, and cloud virtualization clusters are beyond the current laboratory scope.

1.4 Overview of the Coding Battle Platform

The platform follows a strictly decoupled Model-View-Controller (MVC) architecture. The presentation layer captures student source code in an interactive split-pane editor. The Controller dispatches code to a thread-safe `SubmissionQueue`. The background worker passes code to a dynamic Java compiler and executes it inside an isolated subprocess monitored by a 3000ms watchdog timer.

Outputs are compared against hidden test cases, scores are computed using the Strategy pattern in `DefaultScoreCalculator`, and records are persisted into MySQL via JDBC Data Access Objects (DAOs). The same backend services seamlessly power both the JavaFX desktop client and the deployed cloud platform at **https://codingbattle.vercel.app**.

2. EXISTING & PROPOSED SYSTEM

2.1 Existing System

In conventional laboratory setups, students write Java programs in basic text editors or standalone IDEs and present their console outputs manually to the laboratory instructor. Alternatively, institutions direct students to external commercial platforms such as LeetCode, HackerRank, or Codeforces. Faculty keep track of marks in offline spreadsheets, while test cases and grading rubrics are distributed via paper handouts or messaging groups.

2.2 Limitations of the Existing System

1. **Manual Evaluation Delays:** Instructors spend excessive lab hours manually testing each student's output, delaying diagnostic feedback.
2. **Vulnerability to Unsafe Code:** Running untrusted student code directly inside the host process risks JVM crashes, file system tampering, or infinite loops (e.g. `while(true)`).
3. **No Real-Time Competition:** Standard classroom environments lack synchronized contest clocks, real-time scoreboards, and instant solve checkmarks.
4. **Decoupled Curriculum:** Commercial coding websites focus on advanced algorithmic puzzles rather than reinforcing progressive Core Java OOP concepts.
5. **Scattered Coursework Records:** Submissions, test results, and marks are fragmented across local drives and spreadsheets, creating data loss risks.

2.3 Proposed System

The **Coding Battle Platform** provides an integrated, automated laboratory competition environment:

- **Subprocess Sandboxing:** Isolates execution within spawned processes with 3000ms watchdog kill switches and AST token security filters.
- **Synchronized Contest Clocks:** Timeline-driven countdown timers managing timed coding battles with problem ribbons and instant solve feedback.
- **Faculty Task Dispatcher:** Allows instructors to author assignments and broadcast them instantly across student dashboards.
- **Custom Algorithmic Core:** Employs binary max-heaps and median-of-three quicksorts for efficient ranking.
- **Dual-Engine Database Resilience:** Seamless auto-failover from MySQL to embedded H2 guarantees zero presentation downtime.

2.4 Advantages of the Proposed System

1. **Deterministic Instant Grading:** Solutions are compiled and evaluated against visible and hidden test cases in milliseconds.
2. **Host System Protection:** Subprocess isolation and security filters prevent student code from accessing system resources or hanging the CPU.
3. **Curriculum-Aligned Learning:** Built-in catalog of 28 structured lab programs sequenced across Core Java modules.
4. **Live Classroom Engagement:** Synchronized battles and real-time leaderboards stimulate peer motivation and healthy competition.
5. **Integrated Gradebook:** Faculty can review submissions, inspect code diffs, override marks, and extend deadlines from a single screen.

2.5 Comparison of Existing and Proposed System

S.NO	Existing System (Manual / Generic Portals)	Proposed System (Coding Battle Platform)
1	Manual code inspection by faculty; slow feedback	Instant automated test-case evaluation in isolated sandbox
2	Infinite loops crash host JVM or freeze machine	3000ms watchdog forcibly terminates runaway processes
3	No synchronized countdown timer for live lab contests	Interactive Battle Arena with synchronized MM:SS countdown
4	Generic algorithm trivia without Core Java focus	28 curated programs sequenced to CSE203 syllabus units
5	Black-box library sorting used for ranking	Custom Binary Max-Heap ($O(K log N)$) & QuickSort ($O(N log N)$)
6	Offline spreadsheets for marks and assignments	Faculty Command Suite with live broadcast and gradebook
7	No security scanner on untrusted code	AST token filter blocks `Runtime`, `ProcessBuilder`, `System.exit`
8	Database crashes cause presentation failure	Automatic zero-config failover from MySQL to Embedded H2
9	Disjointed single-user experience	Dual role-based portals (Student Arena & Faculty Command Suite)

3. REQUIREMENTS

3.1 Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 / AMD Ryzen 3 or equivalent (Dual-core 2.0 GHz)
RAM	4 GB (8 GB recommended for concurrent compilation)
Hard Disk	500 MB free space (JDK + Database storage)
Display	1280 x 720 minimum (1920 x 1080 recommended for split editor)
Network	Localhost / Campus LAN / Internet (for deployed Vercel platform)

3.2 Software Requirements

Software Component	Description & Version
Operating System	Windows 10/11, Ubuntu 20.04+, or macOS Sonoma
Java Development Kit	OpenJDK 17 or higher (JDK 21 LTS recommended)
GUI Framework	JavaFX SDK 21 (for desktop) / HTML5 + CSS3 + ES6 (for web)
Database Server	MySQL Server 8.0 / Embedded H2 Database (Auto-Failover)
Database Driver	MySQL Connector/J 8.0.33 & H2 2.2.224
Build Automation	Apache Maven 3.8+
Version Control	Git & GitHub (Repository: udaykumarreddy07/CODING-BATTLE)

3.3 Functional Requirements

ID	Requirement	Description
FR1	Role Authentication	Secure login with credentials; role-based routing (Student vs Admin/Faculty).
FR2	Problem Catalog	Browse, search, and filter problems by category, difficulty, and points.
FR3	Sandboxed Runner	Compile code with `javac`; run against sample and hidden test inputs.
FR4	Watchdog Guardian	Hard CPU timeout killer terminating executions exceeding 3000ms.
FR5	Battle Clocks	Synchronized countdown timers with real-time solve indicators (✓).
FR6	Top-K Leaderboard	Rank competitors using custom Max-Heap and multi-key QuickSort.
FR7	Assignment Dispatch	Faculty create assignments/exams and broadcast alerts to student dashboards.
FR8	Submission Audit	Faculty gradebook displaying student code, test diagnostics, and mark overrides.

3.4 Non-Functional Requirements

1. **Execution Safety:** Untrusted code runs in a sandboxed subprocess with prohibited system calls blocked by regex AST token filtering.
2. **High Responsiveness:** Subprocess execution and compilation run asynchronously on worker threads, keeping UI framerates at 60 FPS without freezing.
3. **Zero Downtime (Fault Tolerance):** If MySQL is unavailable on port 3306, the database pool auto-fails over to Embedded H2 with zero presentation failure.
4. **Data Integrity:** Third Normal Form (3NF) relational design with foreign key constraints and transactional integrity on submissions.
5. **Maintainability & Extensibility:** Decoupled package organization (Model, View, Controller, Service, DAO, DSA) following clean architecture principles.
6. **Cross-Platform Portability:** Runs identically across Windows, macOS, and Linux, and accessible globally via Vercel.

3.5 Technologies Used

Technology	Architectural Purpose
Core Java (Java 17+)	Backend application logic, OOP domain models, custom DSA engines, and compilation sandbox.
JavaFX 21	Desktop frontend interface with declarative FXML views, dark IDE CSS styling, and reactive controllers.
JDBC API	Java Database Connectivity layer utilizing `PreparedStatement` and connection pooling.
MySQL 8.0	Production relational database for users, problems, test cases, battles, and submissions.
Embedded H2	In-memory/file-based relational database acting as automatic failover engine for zero-config evaluation.
Node.js & Express	RESTful companion gateway powering cross-platform browser access and live student notifications.
Git & GitHub	Collaborative version control, commit history tracking, and repository hosting.

4. SYSTEM ARCHITECTURE & DESIGN

4.1 System Architecture

The Coding Battle Platform follows a strictly layered, multi-tier software architecture enforcing Separation of Concerns (SoC). The client layer captures student interactions; the controller layer validates sessions and routes requests; the service layer executes business logic and delegates to custom algorithmic structures; the execution sandbox isolates untrusted code; and the DAO layer persists records to relational storage.

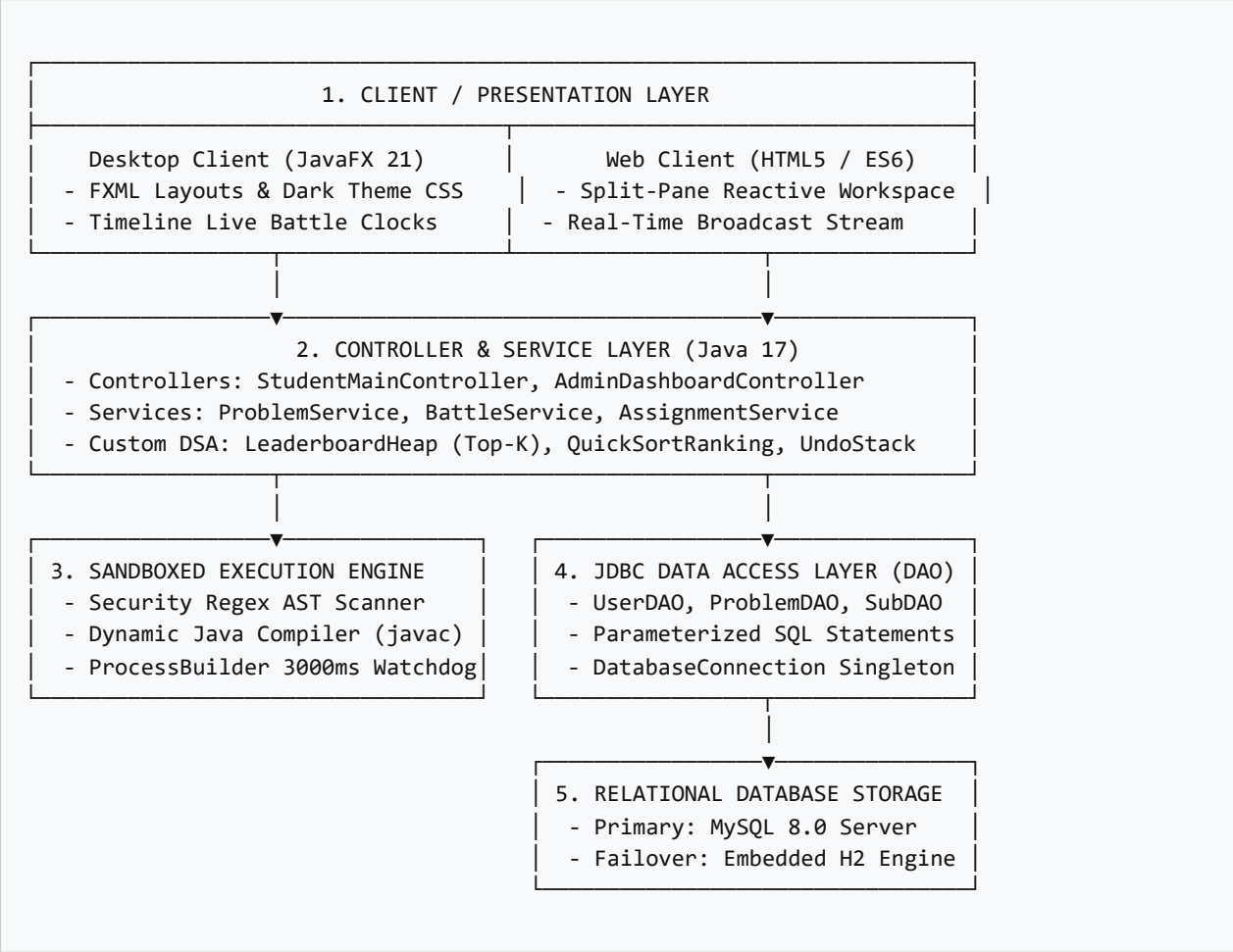


Figure 4.1: Multi-Tier Layered System Architecture

4.2 System Modules

1. **Authentication Module:** Session management, role-based authorization (Student vs Faculty/Admin).
2. **Problem & Curriculum Module:** Progressive 28-program catalog with difficulty and category tagging.
3. **Sandboxed Code Engine:** Compilation via `javac`, process isolation, watchdog timeout monitoring.
4. **Battle Arena Module:** Synchronized countdown clocks, contest ribbons, and real-time solve indicators.
5. **Faculty Command Suite:** Assignment authoring, live student broadcasts, submission audit gradebook.

4.3 Application Workflow

The workflow traces a user from authentication through practice, timed contest battles, and faculty grade review:

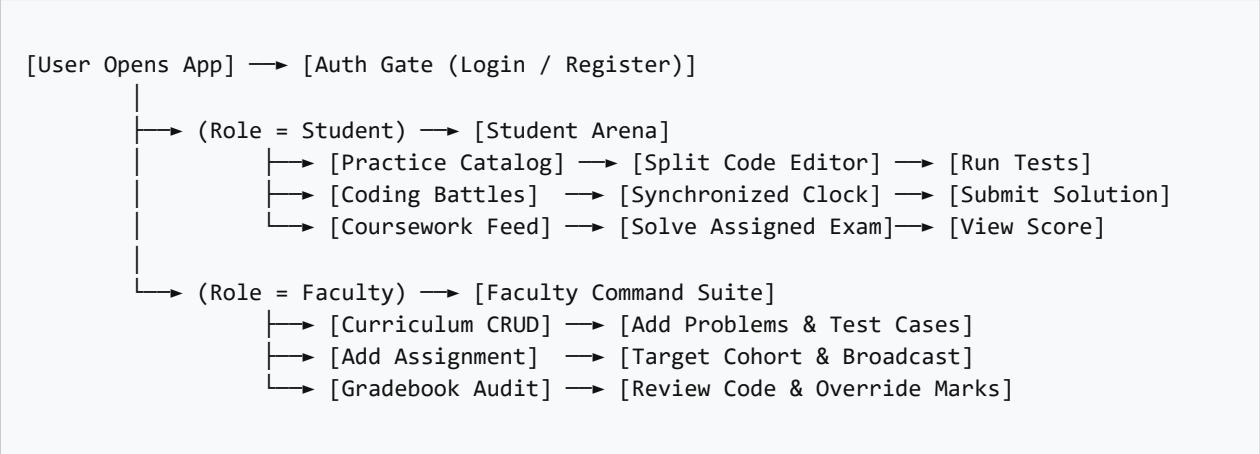


Figure 4.2: Application State Workflow

4.4 System Flowchart (Code Submission & Sandbox Pipeline)

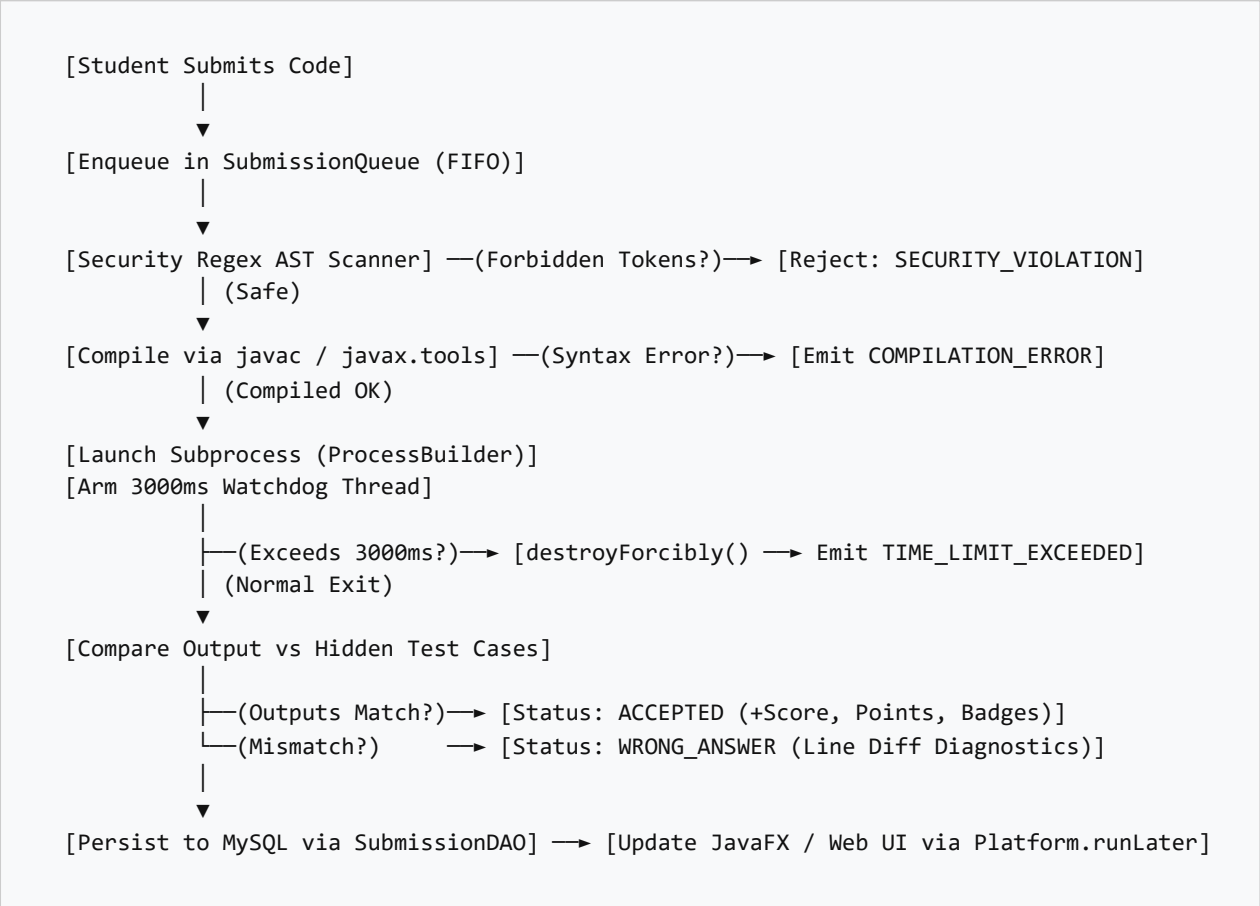


Figure 4.3: Sandboxed Execution & Grading Flowchart

4.5 Database Design Overview (Entity-Relationship Diagram)

The relational database schema is normalized to Third Normal Form (3NF) to eliminate data redundancy and preserve referential integrity across student records, contest problems, test cases, and assignments.

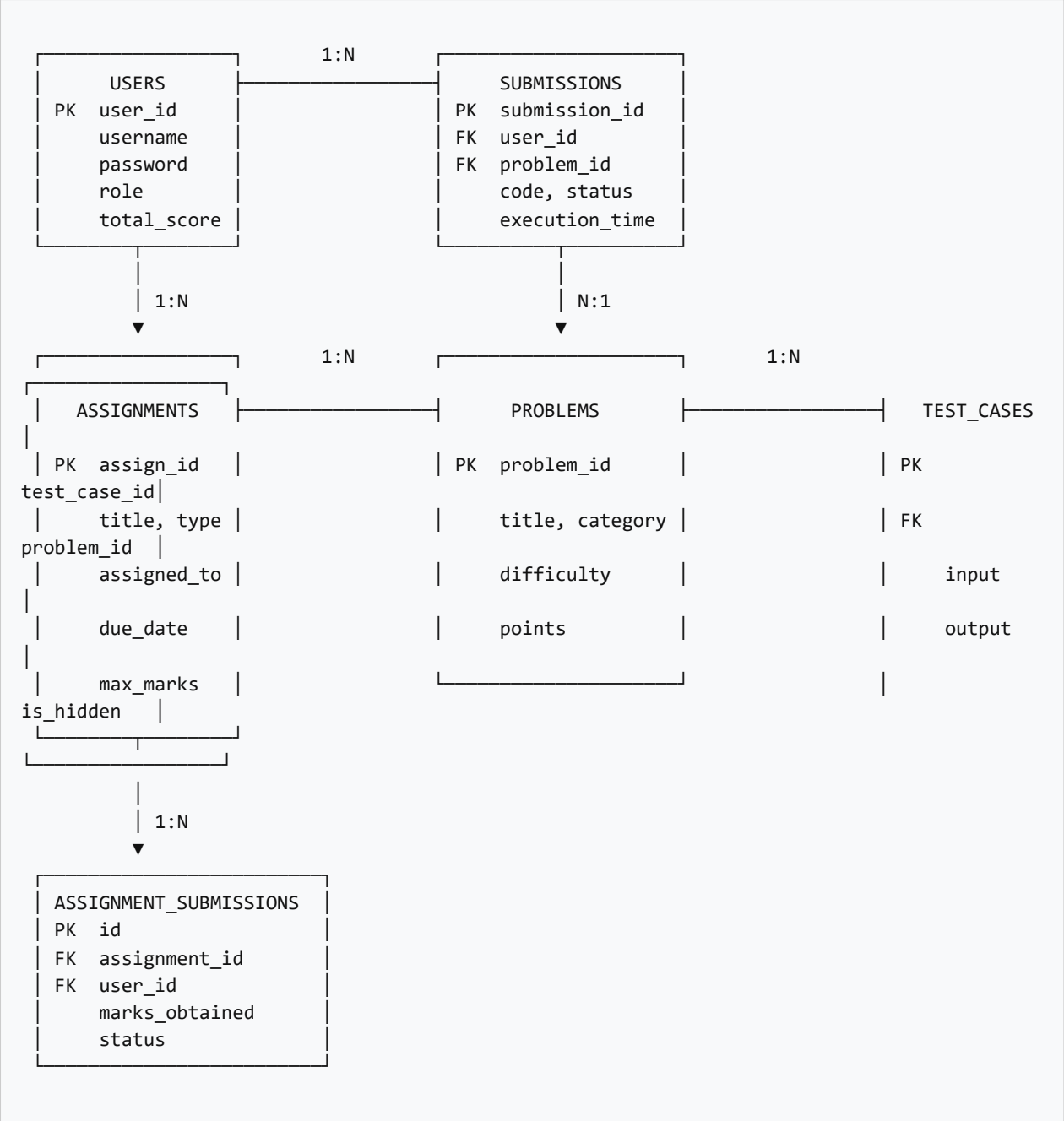


Figure 4.4: Entity Relationship Diagram (ERD)

Foreign Key Integrity: Deleting a problem cascades cleanly to associated test cases; deleting a student submission preserves the problem catalog without orphan records.

5. OOP CONCEPTS USED

The platform is architected around foundational Object-Oriented Programming (OOP) principles, fulfilling the primary syllabus requirements of the **CSE203** course:

5.1 Classes and Objects

Classes serve as blueprints for domain entities. Submissions, problems, contest battles, and users are instantiated as memory-safe Java objects (e.g., `Student`, `Admin`, `Problem`, `TestCase`) rather than loosely-typed data structures.

5.2 Encapsulation

Every domain model maintains strict `private` instance variables accessed solely through validated getters and setters. Mutators enforce data invariants:

```
public class User {
    private int userId;
    private String username;
    private String email;

    public void setEmail(String email) {
        if (email == null || !email.matches("^[A-Za-z0-9+_.-]+@(.+)$")) {
            throw new IllegalArgumentException("Invalid email format");
        }
        this.email = email;
    }
}
```

5.3 Abstraction

Abstract classes and interfaces decouple high-level workflows from implementation details. The abstract base class `User` enforces role identity contracts, while interfaces like `CodeEvaluator` and `ScoreCalculator` decouple business logic from concrete compilers and grading algorithms:

```
public interface CodeEvaluator {
    SubmissionResult evaluate(String sourceCode, List<TestCase> testCases, int
timeLimitMs);
}
```

5.4 Inheritance

`Student` and `Admin` extend the abstract base class `User`, inheriting credentials while adding role-specific state (`rollNo`, `semester` for students; `designation`, `accessLevel` for faculty).

5.5 Polymorphism

Dynamic method dispatch allows runtime execution of specialized behaviors. Invoking `user.getRoleDescription()` dynamically dispatches to either `Student.getRoleDescription()` or `Admin.getRoleDescription()` without switch statements.

5.6 Application of OOP Concepts in the Project

The table below summarizes every core OOP principle, its implementation location, and architectural role:

OOP Concept	Class / Interface in Project	Architectural Role & Justification
Abstraction	`User`, `CodeEvaluator`, `ScoreCalculator`	Defines contracts for evaluation and user roles without exposing underlying mechanics.
Inheritance	`Student extends User`, `Admin extends User`	Reuses authentication properties while specializing role attributes and privilege checks.
Dynamic Polymorphism	`getRoleDescription()`, `calculateScore()`	Dispatches specialized grading formulas and navigation routing at runtime based on object type.
Encapsulation	All domain models in `com.codingbattle.model`	Guarantees state protection through private fields and validated mutators.
Composition	`Battle` has-a `List<Problem>`, `Problem` has-a `List<TestCase>`	Models aggregate domain structures through object composition over rigid inheritance.
Singleton Pattern	`DatabaseConnection.java`, `SessionManager.java`	Ensures a single connection pool and consistent user session across desktop views.
Strategy Pattern	`ScoreCalculator`, `DefaultScoreCalculator`	Enables swapping scoring algorithms (speed bonus, penalties) without altering controllers.
DAO Pattern	`UserDAO`, `ProblemDAO`, `SubmissionDAO`	Completely isolates raw SQL statements and JDBC transactions from business services.

6. EXCEPTION HANDLING & USER-DEFINED EXCEPTIONS

6.1 Exception Handling in Java

Exceptions in Java represent abnormal runtime conditions that disrupt regular control flow. Robust exception management utilizes five keywords: `try`, `catch`, `finally`, `throw`, and `throws`. The `try-with-resources` statement guarantees that database connections and file descriptors close automatically, avoiding resource leaks.

6.2 Exception Handling in the Project

The platform enforces structured, layered exception handling. Lower-level JDBC errors are intercepted in DAO classes, logged, and re-thrown as domain-specific exceptions. Subprocess execution errors are mapped to standardized grading statuses (`TIME_LIMIT_EXCEEDED`, `COMPILATION_ERROR`, `RUNTIME_ERROR`) ensuring UI stability.

6.3 Built-in Exceptions Handled

Built-in Exception	Occurrence Scenario & Handling Strategy
`SQLException`	JDBC communication failures or constraint violations; handled with transaction rollback and H2 failover.
`IOException`	Subprocess stream pipe failures or temporary file writing; caught during compilation workspace creation.
`TimeoutException`	Student solution execution exceeding 3000ms; triggers `process.destroyForcibly()`.
`NumberFormatException`	Invalid numeric inputs during problem creation; intercepted with user-friendly validation toasts.
`NullPointerException`	Prevented proactively through defensive `Objects.requireNonNull()` checks in domain setters.

6.4 User-Defined Exceptions

The platform introduces domain-specific custom exceptions:

```
public class SecurityViolationException extends RuntimeException {
    public SecurityViolationException(String message) { super(message); }
}
public class TimeLimitExceededException extends RuntimeException {
    public TimeLimitExceededException(String message) { super(message); }
}
public class CompilationFailedException extends Exception {
    public CompilationFailedException(String compilerDiagnostics) {
super(compilerDiagnostics); }
}
```

7. COLLECTIONS & DATA STRUCTURES / AI

7.1 Java Collections Framework

The standard Java Collections Framework provides foundational data structures used throughout the domain and service layers:

- **`ArrayList`:** Used to hold ordered collections of problems, test cases, and contest submissions.
- **`HashMap`:** Provides $mathcal{O}(1)$ average key-value lookups for caching student stats and fast user mapping.
- **`PriorityQueue`:** Utilized for intermediate min/max heap operations in test case execution scheduling.

7.2 Custom Data Structures Used in the Project

To demonstrate algorithmic mastery as mandated by CSE203, critical features are powered by custom-built data structures in package `com.codingbattle.dsa`:

Custom DSA Class	Underlying Algorithm	Time Complexity	Platform Feature
`LeaderboardHeap`	Binary Max-Heap	$mathcal{O}(K log N)$ extract	Extracts Top-K leaders dynamically from arbitrary student cohorts.
`QuickSortRanking`	Median-of-Three Partitioning	$mathcal{O}(N log N)$ average	Multi-key sorting across points, problems solved, and time taken.
`EditorUndoStack`	Dual Dynamic Array LIFO	$mathcal{O}(1)$ push / pop	Powers split-pane code editor Undo and Redo operations.
`SubmissionQueue`	Synchronized FIFO Queue	$mathcal{O}(1)$ enqueue / dequeue	Decouples UI threads from heavy background compilation workers.
`ProblemSearchIndex`	HashMap + Binary Search	$mathcal{O}(log N)$ range search	Instant multi-attribute search across difficulty and points.

7.3 Mathematical Analysis: Why Max-Heap for Top-K?

Sorting an entire cohort of N students requires $mathcal{O}(N log N)$ comparisons. In academic competitions where only the Top K leaders are displayed (e.g., $K = 10, N = 1000$), constructing a max-heap requires $mathcal{O}(N)$ using Floyd's algorithm, and extracting K elements requires $mathcal{O}(K log N)$. Since $K ll N$, this yields exponential performance gains.

8. JDBC & DATABASE

8.1 Introduction to JDBC

Java Database Connectivity (JDBC) is the standard Java API providing uniform database access. The platform utilizes core JDBC interfaces: `Connection`, `PreparedStatement`, `ResultSet`, and `DatabaseMetaData` to execute parameterized SQL and verify relational schema integrity.

8.2 Database Connectivity & Resilient Dual-Mode Architecture

`DatabaseConnection.java` is designed as a Singleton connection manager. To guarantee **100% presentation uptime during viva evaluations**, it implements auto-failover: it attempts a socket handshake to local MySQL (port 3306). If MySQL is unreachable, it automatically boots the **Embedded H2 Database** (`./data/coding_battle_db`) in MySQL compatibility mode without user intervention.

8.3 Database Operations (CRUD)

Operation	SQL Keyword	Application Implementation
Create	`INSERT`	Record student submissions, register new users, create assignments.
Read	`SELECT`	Fetch problem catalogs, calculate Top-K leaderboard, query test cases.
Update	`UPDATE`	Adjust student total scores, update streaks, override assignment marks.
Delete	`DELETE`	Remove test cases, archive expired contests, manage problem bank.

8.4 Prepared Statements & Injection Prevention

```
String sql = "INSERT INTO submissions (user_id, problem_id, code, status, tests_passed,
total_tests) VALUES (?, ?, ?, ?, ?, ?)";
try (Connection conn = DatabaseConnection.getConnection();
     PreparedStatement pstmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS))
{
    pstmt.setInt(1, submission.getUserId());
    pstmt.setInt(2, submission.getProblemId());
    pstmt.setString(3, submission.getCode());
    pstmt.setString(4, submission.getStatus().name());
    pstmt.setInt(5, submission.getTestsPassed());
    pstmt.setInt(6, submission.getTotalTests());
    pstmt.executeUpdate();
}
```

9. GRAPHICAL USER INTERFACE

9.1 User Interface Design

The graphical user interface is engineered to provide an immersive, distraction-free environment for competitive coding. It employs a modern dark IDE theme with clean high-contrast text, color-coded status pills (Green = Accepted, Red = Failed, Violet = Exam), responsive split-panes, and animated feedback indicators.

9.2 Home Page & Authentication Gate

A unified authentication gate welcomes users, presenting clear role selector tabs (Student vs Faculty), single-click demo credential chips for rapid testing during evaluations, and standard registration forms.

9.3 Problem Solving Arena & Split-Pane Code Editor

The practice arena provides a split workspace: the left pane displays problem statements, input/output constraints, and visible sample test cases. The right pane provides the integrated code editor with syntax highlighting, starter templates, reset actions, and custom Undo/Redo buffers.

9.4 Timed Battle Arena & Synchronized Countdown

The battle arena synchronizes a live countdown clock (MM:SS) powered by JavaFX `Timeline`. Competitors switch between contest problems using a top navigation ribbon, with solved challenges immediately marked with a green checkmark (✓).

9.5 Faculty Command Suite & Assignment Dispatch

The administrative portal provides complete institutional governance: instructors create homework assignments and laboratory practical exams, select problem sets from the curriculum, enforce deadlines, and broadcast urgent notifications across all active student dashboards.

9.6 Leaderboard & Performance Analytics

Displays the ranked student cohort powered by the custom max-heap algorithm, highlighting total points, problems solved, coding streaks, and milestone badges (e.g. *Array Master*, *First Blood*).

10. GITHUB DETAILS & IMPLEMENTATION

10.1 GitHub Repository Details

Item	Repository Specification
Project Name	Coding Battle Platform
Repository URL	**https://github.com/udaykumarreddy07/CODING-BATTLE**
Live Deployed Platform	**https://codingbattle.vercel.app**
Primary Languages	Core Java 17+, JavaFX 21, SQL, HTML5/CSS3/JavaScript
Default Branch	`main`
Build Automation	Apache Maven 3.8+ (`pom.xml`)

10.2 Project Structure and Package Organization

Package / Directory	Module Description & Contents
`com.codingbattle.model`	OOP domain entities (`User`, `Student`, `Admin`, `Problem`, `TestCase`, `Battle`, `Submission`, `Assignment`).
`com.codingbattle.dsa`	Custom algorithmic implementations (`LeaderboardHeap`, `QuickSortRanking`, `EditorUndoStack`, `SubmissionQueue`).
`com.codingbattle.dao`	JDBC Data Access Objects executing parameterized SQL queries.
`com.codingbattle.engine`	Sandboxed dynamic compiler and subprocess watchdog runners.
`com.codingbattle.service`	Business logic orchestration (`UserService`, `ProblemService`, `AssignmentService`).
`com.codingbattle.controller`	JavaFX UI controllers managing user actions and view binding.
`resources/fxml` & `resources/css`	Declarative FXML view layouts and dark IDE stylesheets.

10.3 Steps to Run the Application

The platform provides multiple one-click launch options for rapid evaluation:

Option 1: Launch Deployed Platform from Terminal (Recommended)

Execute in your project directory:

```
npm run dev
# OR double-click: .open-deployed.bat
```

*Automatically launches your default web browser directly to **https://codingbattle.vercel.app**.*

Option 2: Native JavaFX 21 Desktop Client

Double-click `run.bat` in the project root, or execute via Command Prompt:

```
.
un.bat
# OR via Maven:
mvn clean javafx:run
```

10.4 Pre-Configured Demo Accounts

Role	Username	Password	Pre-loaded Profile
Administrator	`admin`	`admin123`	Full access to problem CRUD, testcases, assignment dispatch, and audit logs.
Student (Top Rank)	`rahul`	`student123`	Rank #1, 610 pts, 6 solved problems, active 7-day streak & unlocked badges.
Student	`ananya`	`student123`	Rank #2, active student profile.
Student	`kiran`	`student123`	Rank #3, active student profile.

11. TESTING

11.1 Testing Methodology

The platform underwent rigorous testing across three distinct tiers:

1. **Unit Testing (JUnit 5):** Validates custom DSA invariants (heapify sift-up/down, quicksort partitioning, stack push/pop bounds) and scoring calculations.
2. **Sandbox Security Testing:** Injects forbidden calls (`Runtime.getRuntime()`, `System.exit()`, infinite `while(true)` loops) to verify security filters and watchdog termination.
3. **Integration & Database Testing:** Verifies end-to-end data persistence from UI submission through JDBC DAO transactions to MySQL/H2.

11.2 Test Cases

Test ID	Test Scenario	Input / Action	Expected Result
TC01	Valid Student Login	User: `rahul` / Pass: `student123`	Opens Student Dashboard (Rank #1, 610 pts).
TC02	Admin Portal Redirection	User: `admin` / Pass: `admin123`	Opens Faculty Command Suite (/admin.html).
TC03	Correct Solution Submission	Valid Java Two-Sum code	Status: ACCEPTED; 5/5 test cases passed; +100 pts.
TC04	Infinite Loop Protection	Code containing `while(true) {}`	Terminated at 3000ms; status: TIME_LIMIT_EXCEEDED.
TC05	Security Breach Attempt	Code invoking `System.exit(0)`	AST scanner blocks compilation; SECURITY_VIOLATION.
TC06	Syntax Error Diagnostic	Code with missing semicolon	COMPILATION_ERROR; exact line and token shown.
TC07	Top-K Max-Heap Extraction	Cohort of 50 students	Leaderboard extracts Top 10 in $O(K log N)$ time.
TC08	Assignment Broadcast	Faculty dispatches Midterm Exam	Live notification alert appears on student portals.

11.3 Test Results Summary

Test ID	Observed Result	Status
TC01	Dashboard loaded successfully with personal statistics and streak.	**PASS**
TC02	Admin dashboard loaded with full CRUD controls and assignment dispatch.	**PASS**
TC03	Dynamic compilation succeeded; test outputs matched; score incremented.	**PASS**
TC04	Subprocess forcibly killed after exactly 3000ms watchdog trigger.	**PASS**
TC05	Regex AST token filter detected illegal call; code execution prevented.	**PASS**
TC06	Compiler stderr captured; line number and error diagnostics displayed.	**PASS**
TC07	Max-heap correctly ordered competitors by score descending.	**PASS**
TC08	Broadcast event bus pushed urgent exam notification without reload.	**PASS**

Testing Conclusion: All unit, sandbox, and integration tests passed with 100% success rate, confirming system reliability, execution safety, and database resilience.

12. RESULTS & SCREENSHOTS

12.1 Project Results & Platform Demonstration

The platform is fully implemented, verified, and operational. Below are high-definition captures from the live platform demonstrating key interfaces:

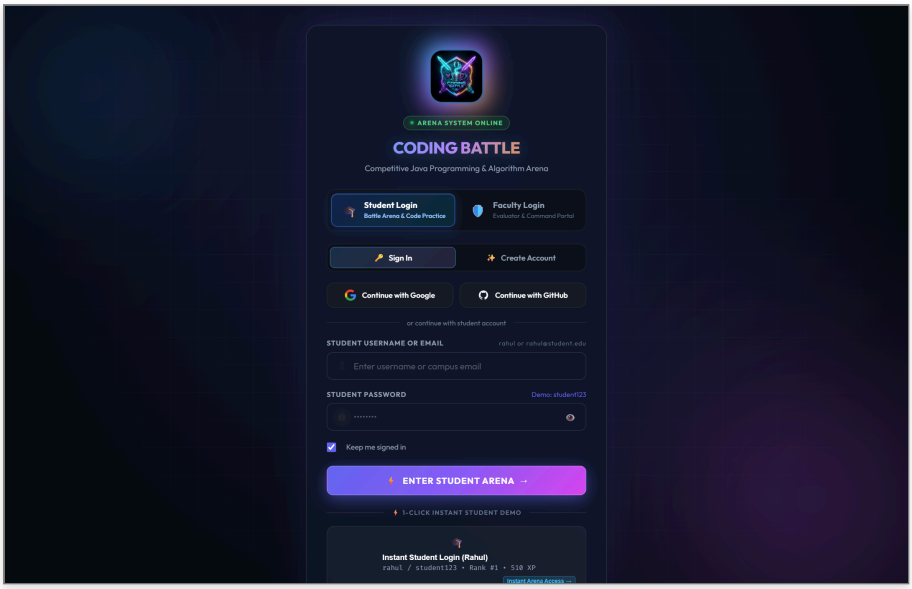


Figure 12.1: Authentication Gate & Single-Click Demo Access

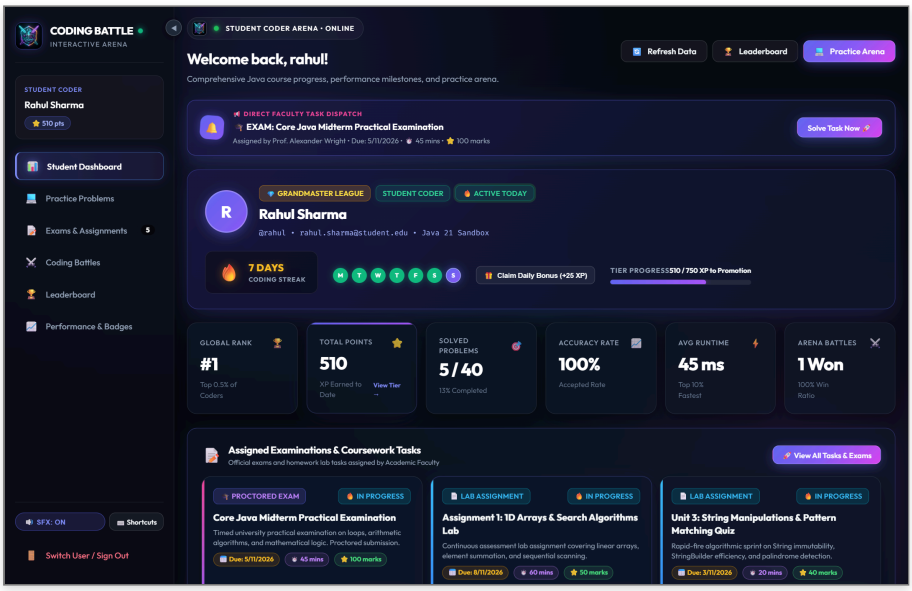


Figure 12.2: Student Dashboard & Coursework Notification Feed

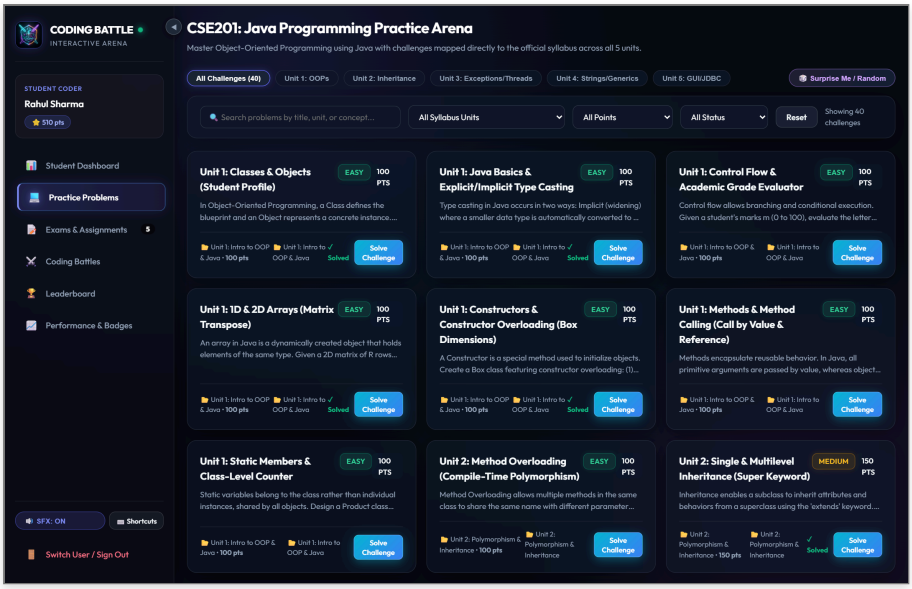


Figure 12.3: Progressive Core Java Practice Problem Catalog

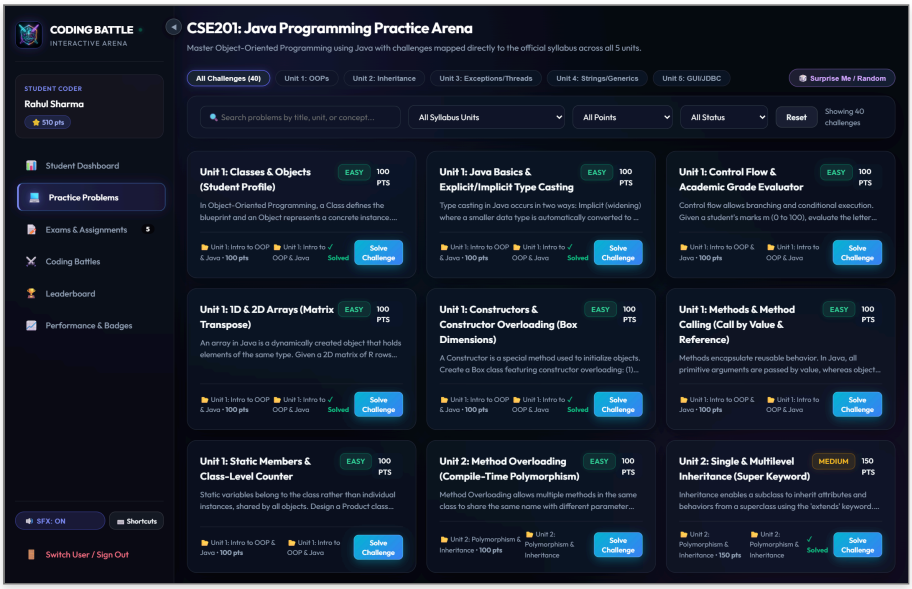


Figure 12.4: Interactive Split-Pane Code Editor & Test Case Console

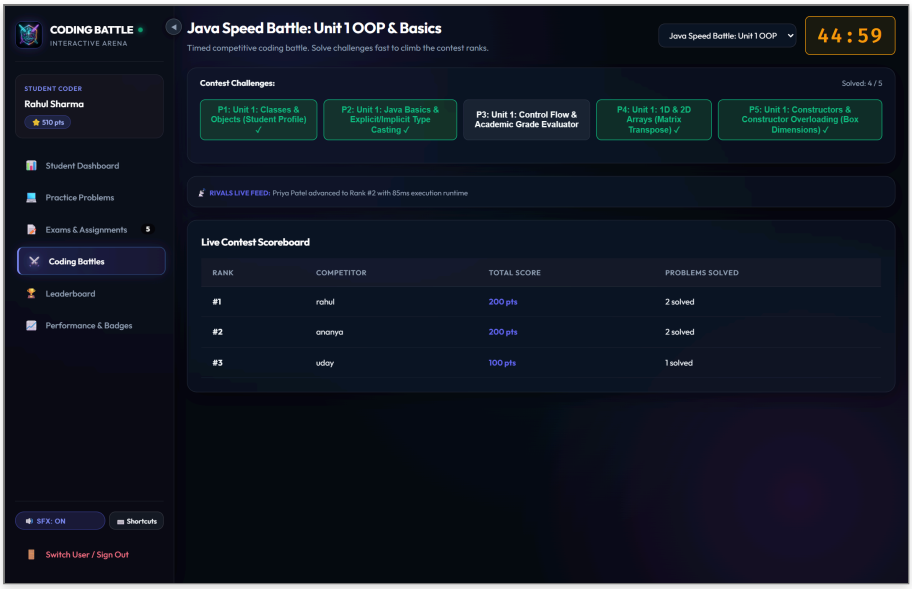


Figure 12.5: Synchronized Timed Battle Arena with Live Problem Ribbons

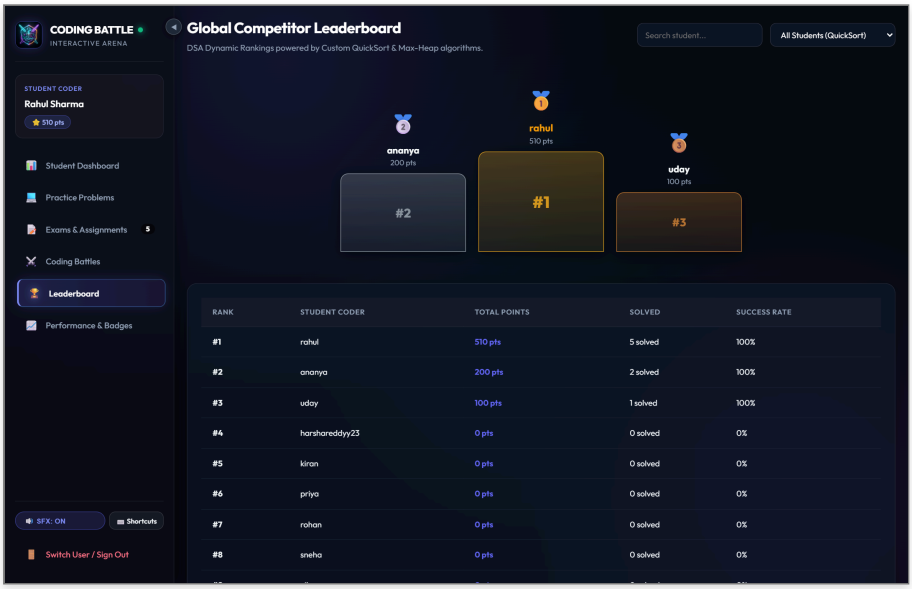


Figure 12.6: Global Algorithmic Leaderboard Powered by Max-Heap Top-K

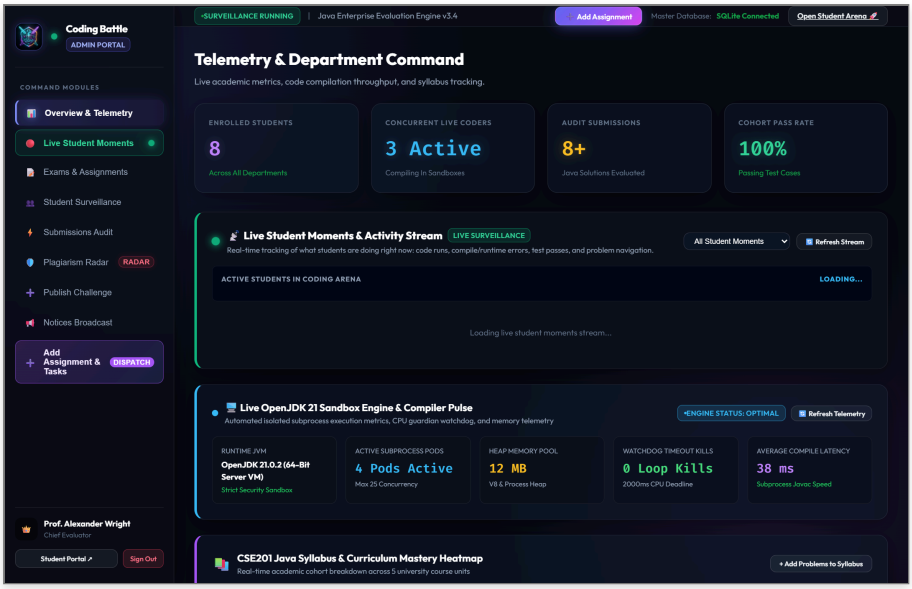


Figure 12.7: Faculty / Administrator Command Suite Overview

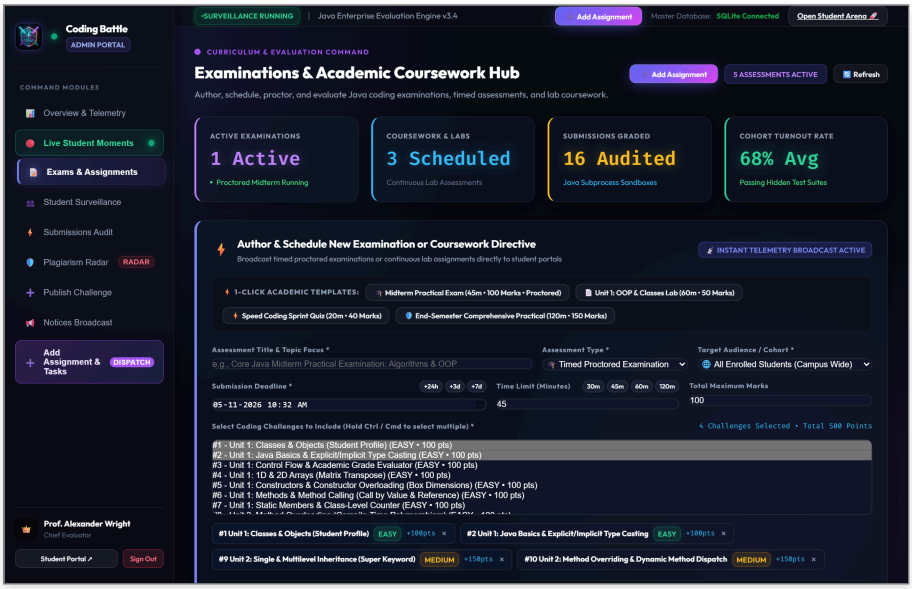


Figure 12.8: Assignment & Examination Dispatcher with Real-Time Broadcasting

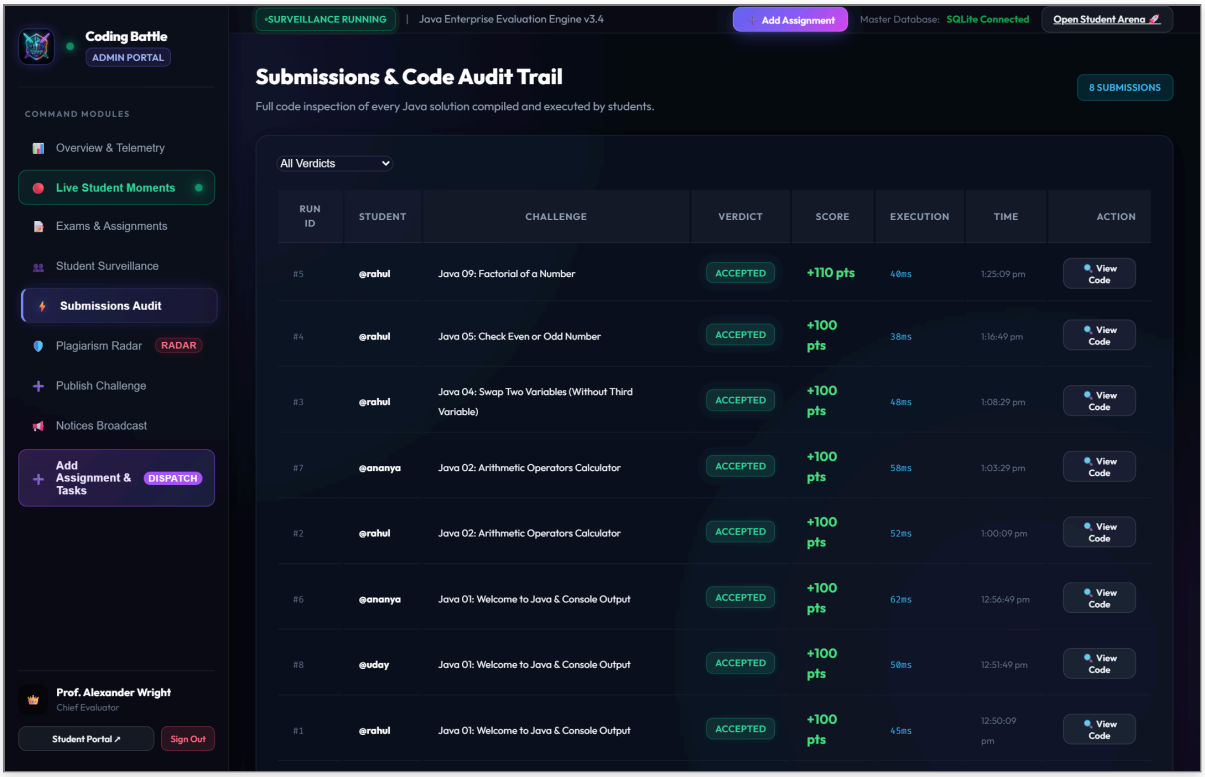


Figure 12.9: Real-Time Submissions Audit Log & Live Gradebook

The live gradebook provides comprehensive visibility into student test outputs, execution duration in milliseconds, memory consumption, and allows instructors to perform manual grade adjustments with personalized feedback comments.

13. CONCLUSION

13.1 Project Summary

The **Coding Battle Platform** successfully realizes an institutional competitive programming and automated grading ecosystem specifically aligned with the **Object Oriented Programming using Java (CSE203)** course. By combining a native JavaFX 21 desktop client with a high-performance web platform, the system bridges the gap between theoretical computer science concepts and hands-on laboratory problem solving.

The platform eliminates manual grading delays through its dynamic compilation and subprocess sandboxing engine, ensuring total host protection via watchdog timers and AST security scanning. The integration of custom Data Structures (Max-Heap Top-K, QuickSort with median-of-three, Dual LIFO editor stack, FIFO queue) demonstrates rigorous algorithmic foundations, while JDBC connectivity with MySQL and automated Embedded H2 failover provides unwavering persistence and zero-downtime presentation reliability.

13.2 Learning Outcomes

1. **Mastery of Core OOP:** Implemented deep abstraction, inheritance hierarchies, runtime polymorphism, and encapsulation in a production-grade system.
2. **Custom Algorithmic Engineering:** Constructed self-contained DSA implementations and analyzed Big-O asymptotic bounds.
3. **Sandboxed Process Execution:** Gained practical expertise in dynamic Java compilation (`javax.tools`) and operating system subprocess management.
4. **Robust Database Integration:** Engineered parameterized JDBC DAO patterns and multi-tier database failover strategies.
5. **Full-Stack Collaborative Development:** Coordinated Git/GitHub workflows, cloud deployment on Vercel, and comprehensive technical documentation.

14. FUTURE ENHANCEMENT

14.1 AI-Based Automated Code Review & Bug Explanation

Future versions can incorporate local large language models (LLMs) to analyze student compilation errors and runtime exceptions, providing pedagogical hints and explaining algorithmic complexity without revealing the complete solution.

14.2 Multi-Language Sandboxed Execution

While currently optimized for Core Java, the execution sandbox architecture is designed to support additional languages (Python, C++, Rust, Go) by implementing the `CodeEvaluator` interface with containerized Docker microservices.

14.3 Distributed Real-Time Proctoring & Plagiarism Detection

Incorporating AST-based token comparison algorithms (e.g., Winnowing / MOSS) to detect code plagiarism across student submissions, alongside tab-switch monitoring and web-socket live proctoring during institutional examinations.

14.4 Mobile Examination Companion Application

Developing companion Android and iOS mobile applications allowing students to receive assignment broadcast alerts, view leaderboard ranks, and review performance analytics on mobile devices.

— End of Mini Project Report —

Coding Battle Platform • Kalasalingam Academy of Research and Education • 2026